

(MSCS)

(SISTEMA DE ASIGNACIÓN DE HORARIOS DE PROFESORES)

Software Design Document

Name (s):

Cuapantecatl Ruiz Raul Irving

Garcia Leon Miguel Angel

Lab Section: Laboratorio computo 3 piso 1

Workstation: Diseño de Software

Date: (25/02/2025)

MSCS



TABLA DE CONTENIDOS

Historial de Revisiones

Fecha	Versión	Autor	Razón del rechazo	Modificación realizada
12/02/25	0.1	Garcia Leon Miguel Angel Cuapantecatl Ruiz Raul Irving		

Historial de Cambio

Versión	Fecha y Horas invertidas	Revisado por	Roll
0.1	12/02/25 - 01:30 Hrs		

Historial de elaboración del documento

Versión	Fecha de Revisión	Realizado por	Actualización se agrega
0.1	12/02/25	Garcia Leon Miguel Angel Cuapantecatl Ruiz Raul Irving	
0.1	25/02/25	Cuapantecatl Ruiz Raul Irving	introducción y casos de uso
0.1	25/02/25	Cuapantecatl Ruiz Raul Irving	se realiza diagrama de contexto
0.1	05/03/25	Cuapantecatl Ruiz Raul Irving	Se anexa diagramas en cada caso de uso
0.1	06/03/25	Cuapantecatl Ruiz Raul Irving	Se anexa diagramas en cada caso de uso faltantes
0.1	13/03/25	Garcia Leon Miguel Angel	Inclusión de diagrama de paquetes

0.1	13/03/25	Garcia Leon Miguel Angel	Inclusión de diagrama de componentes
0.1	14/03/25	Garcia Leon Miguel Angel	Modificación al diagrama de paquetes
0.1	11/04/25	Cuapantecatl Ruiz Raul Irving	Se realiza Patterns (5.7) Reuse of patterns and available Framework template UML composite structure diagram

1. INTRODUCCIÓN

En esta sección, se presentan los puntos de vista de diseño aplicables al **Sistema de Asignación de Horarios y Materias para Profesores**. Estos puntos de vista proporcionan una representación estructurada del sistema desde diferentes perspectivas, permitiendo una mejor comprensión de sus componentes, relaciones y restricciones.

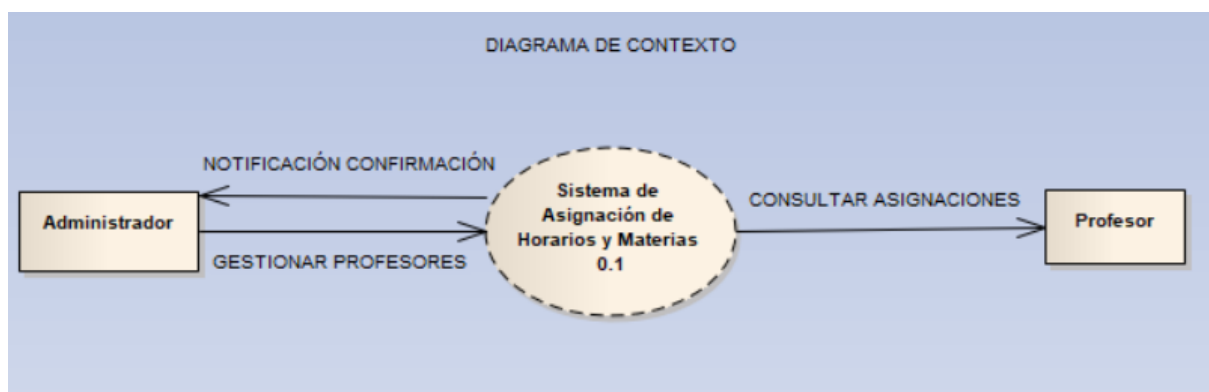
Cada punto de vista está diseñado para abordar preocupaciones específicas del diseño, utilizando lenguajes y notaciones apropiadas para su representación. La aplicación de estos enfoques facilitará la documentación, análisis y validación del sistema en las distintas etapas de su desarrollo.

Contexto:

Descripción General

El Sistema de Asignación de Horarios y Materias para Profesores opera en un entorno donde interactúan dos actores principales: el Administrador y el Profesor. El sistema ofrece servicios específicos para cada uno, facilitando la gestión de horarios, materias y grupos de estudiantes, así como la consulta de información relevante. Definiendo los límites del sistema y su interacción con actores externos mediante diagramas de casos de uso en UML y diagramas de contexto.

Diagrama de Contexto



- **Administrador:** Interactúa con el sistema para gestionar la información relacionada con los profesores, asignaciones y horarios.
- **Profesor:** Utiliza el sistema para consultar su información personal y asignaciones.
- **Sistema de Asignación de Horarios y Materias:** Proporciona los servicios necesarios para la gestión y consulta de información.

Servicios (Casos de Uso)

A continuación, se describen los servicios ofrecidos por el sistema, correspondientes a los casos de uso definidos:

Sistema de Asignación de Horarios y Materias para Profesores

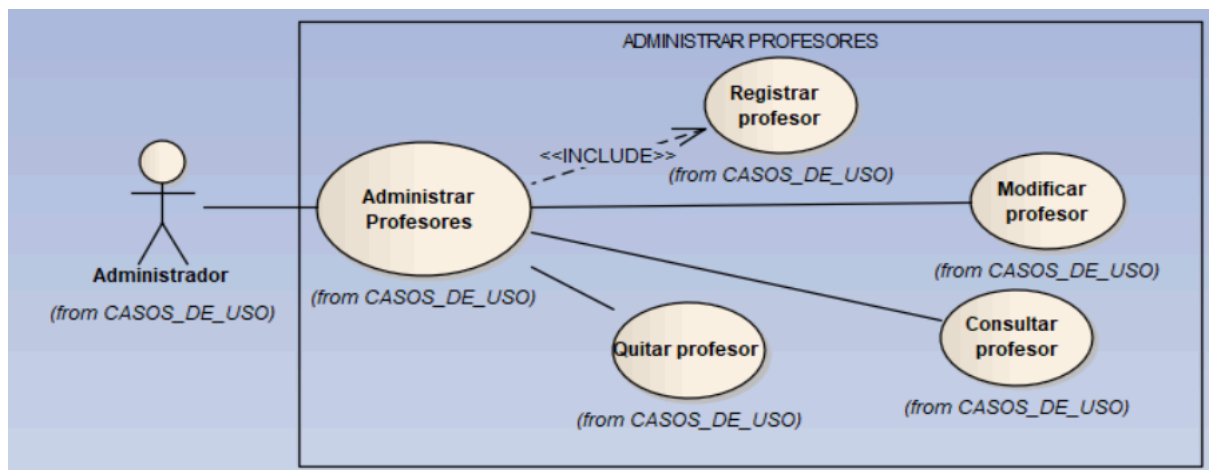
1. Actores del Sistema

Actores Principales:

- **Administrador:** Responsable de la gestión de profesores, asignaciones de materias, horarios y grupos de estudiantes.
- **Profesor:** Usuario que consulta su horario, materias y grupos asignados.

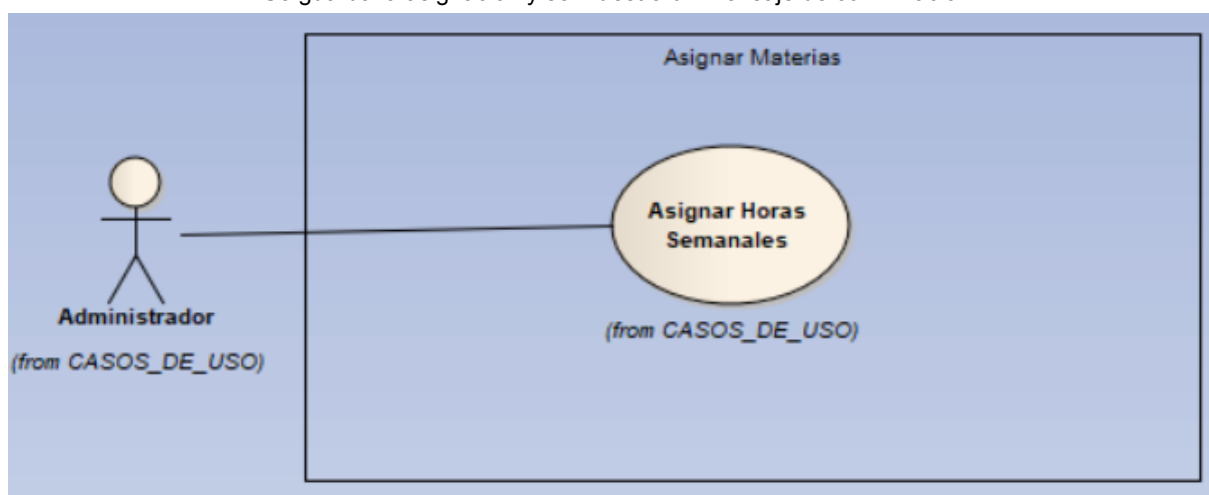
2. Casos de Uso

ID	Caso de uso	Descripción	Actor	Flujo Principal
CU01	Administrar Profesores	Permite al administrador agregar, modificar o eliminar profesores.	Administrador	Evento de activación: El administrador accede al módulo de gestión de profesores para agregar, modificar o eliminar profesores precondición: El administrador debe estar autenticado en el sistema y tener permisos para gestionar profesores. post-condición: La base de datos se actualiza con los nuevos datos modificados o agregados (Nuevo profesor agregado, datos modificados o eliminar profesor) y muestra un mensaje de confirmación.
El administrador accede a la gestión de profesores. Selecciona la opción en el menú que contiene agregar, modificar o eliminar. Ingresa los datos del profesor (nombre, correo, especificación de área, número de empleado.). El sistema valida y almacena la información. Se muestra un mensaje de confirmación.				

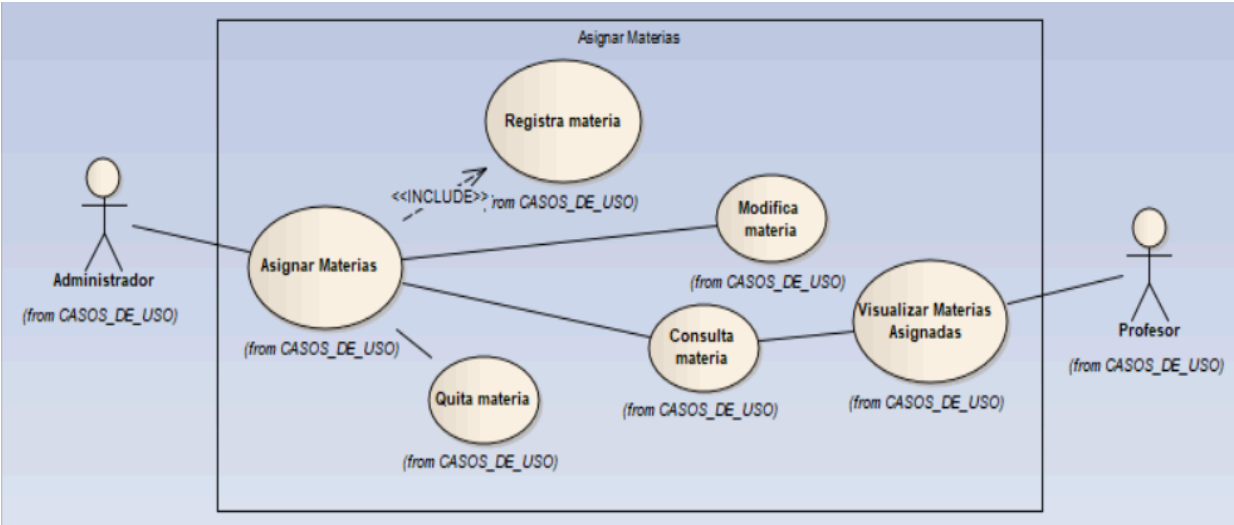


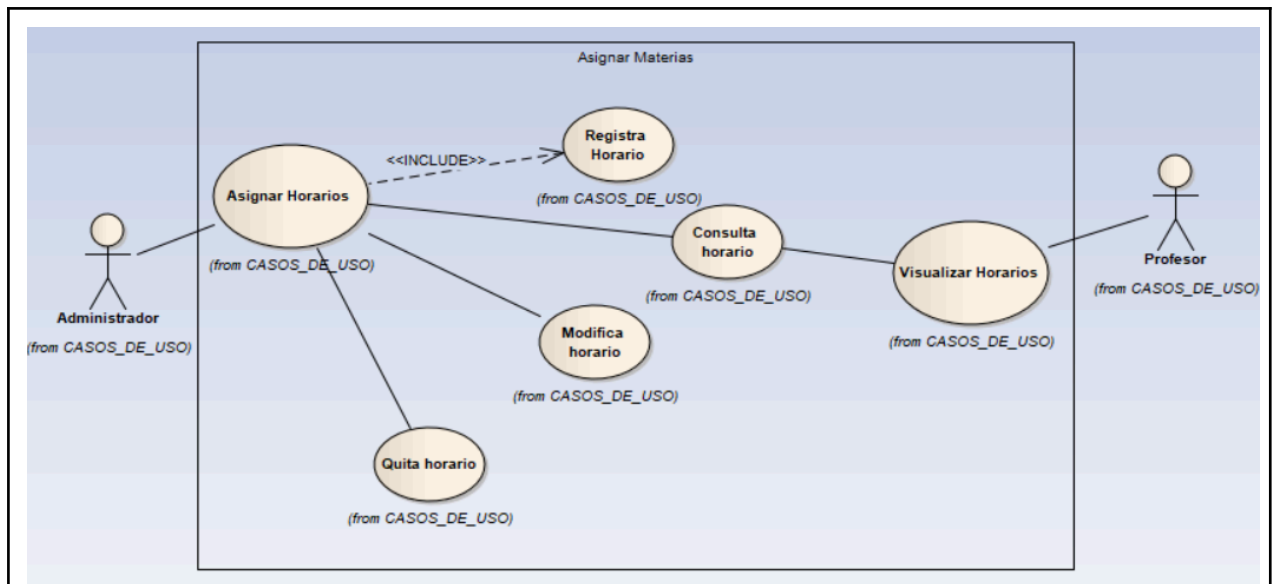
CU02	Asignar Horas Semanales	Permite asignar una cantidad de horas semanales a los profesores según su especialización.	Administrador	Evento de activación: El administrador accede al módulo de asignación de horas, para asignar la cantidad de horas correspondientes a cada profesor. precondición: El administrador debe estar autenticado en el sistema y tener permisos para gestionar profesores, además de que el profesor debe estar agregado ya en el sistema. postcondición: La base de datos se tiene que actualizar con la cantidad de horas asignadas a cada profesor y muestra un mensaje de confirmación
------	-------------------------	--	---------------	--

El administrador accede a la asignación de horas.
 Selecciona un profesor y establece el número de horas.
 El sistema valida que no supere el límite permitido.
 Se guarda la asignación y se muestra un mensaje de confirmación.



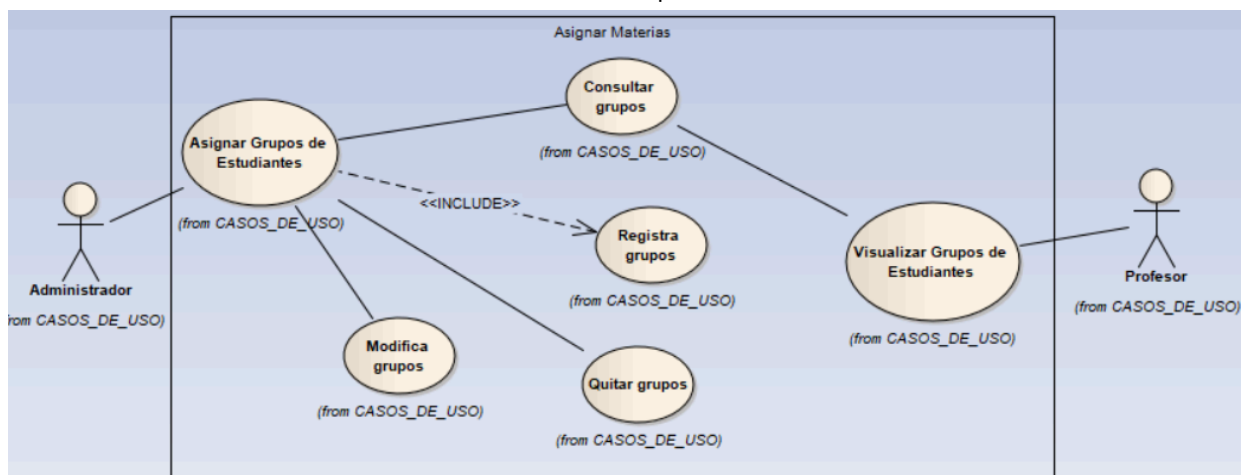
CU03	Asignar Materias	Permite asignar materias a los profesores según su especialización.	Administrador	Evento de activación: El administrador accede al módulo de asignación de materias para asignar una materia a un profesor.
------	------------------	---	---------------	--

				precondición: El administrador debe de estar autenticado en el sistema y tener permisos para la gestión de materias, además de que el profesor tiene que estar registrado y contar con la especialización adecuada para la materia. postcondición: La asignación de la materia se guarda en la base de datos y muestra un mensaje de confirmación.
El administrador accede a la asignación de materias. Selecciona un profesor y la materia correspondiente. El sistema valida y guarda la asignación. Se muestra un mensaje de confirmación.  <pre> graph LR Admin[Administrador] --- AM((Asignar Materias)) AM --> RM((Registra materia)) AM --> MM((Modifica materia)) AM --> CM((Consulta materia)) AM --> QM((Quita materia)) CM --- VMA((Visualizar Materias Asignadas)) VMA --- Prof[Profesor] RM -.-> <<INCLUDE>> from CASOS_DE_USO AM MM -.-> from CASOS_DE_USO AM CM -.-> from CASOS_DE_USO AM VMA -.-> from CASOS_DE_USO AM QM -.-> from CASOS_DE_USO AM </pre>				
CU04	Asignar Horarios	Permite establecer los horarios de cada materia para los profesores.	Administrador	Evento de activación: El administrador accede al módulo de gestión de horarios para establecer los horarios de las materias asignadas a los profesores. precondición: El administrador debe estar autenticado en el sistema y tener permisos para gestionar horarios. El profesor debe tener una materia asignada y el horario no debe generar conflictos con otros eventos del profesor. postcondición: El horario asignado al profesor se guarda en la base de datos y se muestra un mensaje de confirmación.
El administrador accede a la gestión de horarios. Selecciona un profesor y la materia asignada. Define los días y horarios. El sistema valida que no haya conflictos. Se guarda la información y se confirma la operación				



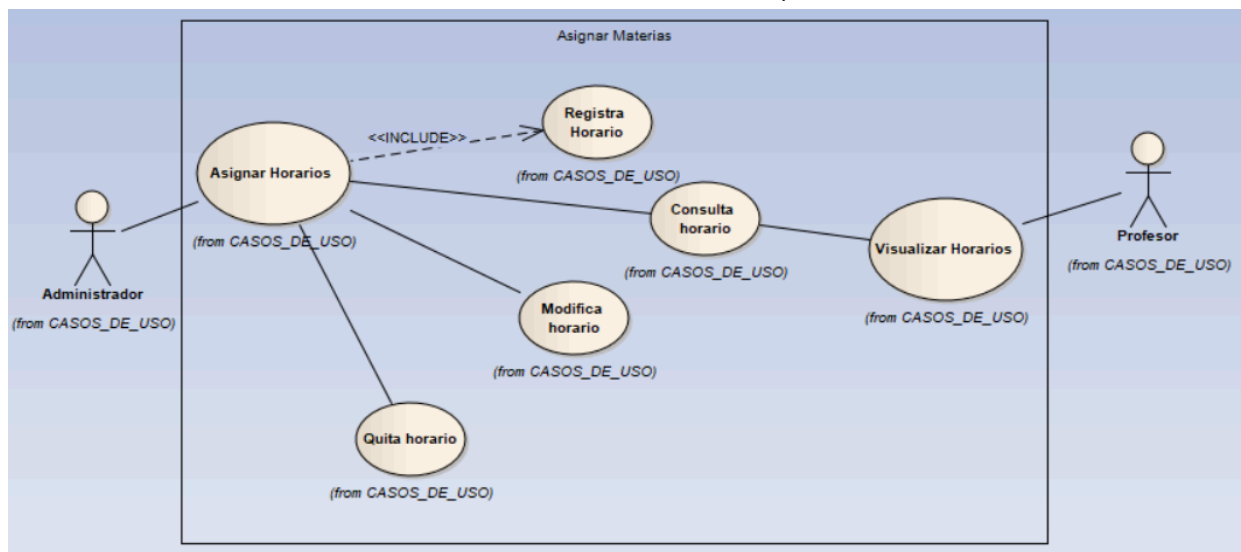
CU05	Asignar Grupos de Estudiantes	Asigna grupos de estudiantes a los profesores para cada materia.	Administrador	Evento de activación: El administrador accede al módulo de asignación de grupos de estudiantes para asignar un grupo a un profesor para una materia precondición: el administrador debe estar autenticado en el sistema y tener permisos para gestionar la asignación de grupos. El profesor debe de tener al menos una materia asignada, y debe de existir al menos un grupo de estudiantes para ser asignado. postcondición: El grupo de estudiantes se asigna al profesor y la materia correspondiente en la base de datos, y se muestra un mensaje de confirmación.
------	-------------------------------	--	---------------	--

El administrador accede a la asignación de grupos.
 Selecciona un profesor y la materia asignada.
 Asigna un grupo de estudiantes.
 El sistema valida y guarda la asignación.
 Se confirma la operación.



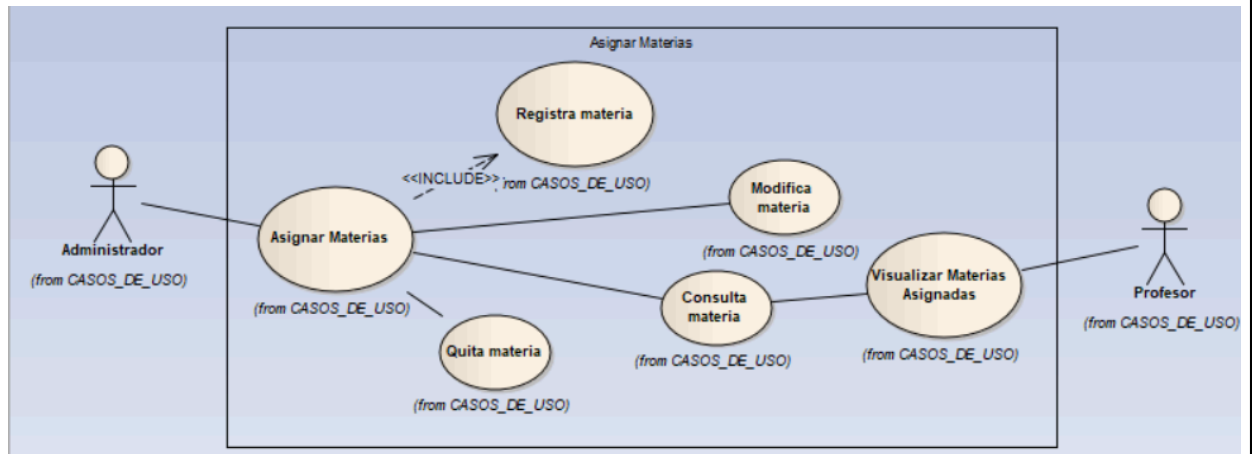
CU06	Visualizar Horarios	Permite a los profesores ver su horario diario y semanal.	Profesor	Evento de activación: El profesor accede al sistema y entra a la sección de horarios. Precondición: El profesor debe estar autenticado en el sistema y tener horarios previamente asignados. Postcondición: El sistema muestra el horario del profesor en formato diario o semanal.
------	---------------------	---	----------	--

El profesor inicia sesión en el sistema.
Accede a la sección de horarios.
Consulta su horario en formato diario o semanal.
El sistema muestra la información correspondiente.



CU07	Visualizar Materias Asignadas	Permite a los profesores ver las materias que deben impartir.	Profesor	Evento de activación: El profesor accede al sistema y entra a la sección de materias asignadas. Precondición: El profesor debe estar autenticado en el sistema y tener al menos una materia asignada. Postcondición: El sistema muestra la lista de materias asignadas con detalles adicionales.
------	-------------------------------	---	----------	---

El profesor accede a su cuenta.
Consulta la sección de materias asignadas.
El sistema muestra la lista de materias con detalles adicionales.



CU08	Visualizar Grupos de Estudiantes	Permite a los profesores conocer los grupos de estudiantes a su cargo.	Profesor	Evento de activación: El profesor accede al sistema y entra a la sección de grupos asignados. Precondición: El profesor debe estar autenticado en el sistema y tener al menos un grupo de estudiantes asignado. Postcondición: El sistema desde la base de datos muestra la lista de grupos de estudiantes asignados al profesor.
------	----------------------------------	--	----------	--

El profesor accede a la plataforma.
 Consulta la sección de grupos asignados.
 El sistema muestra la lista de grupos

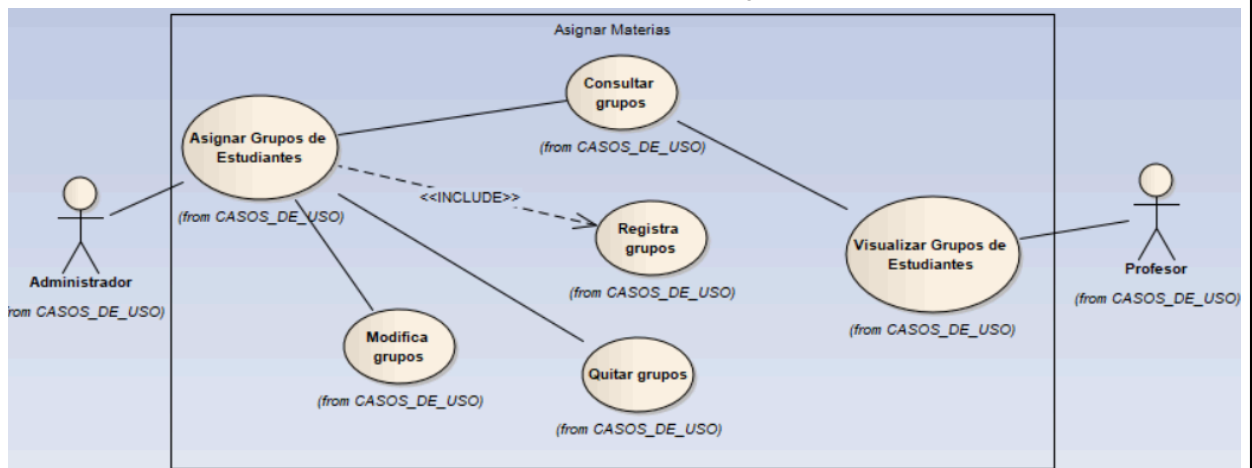
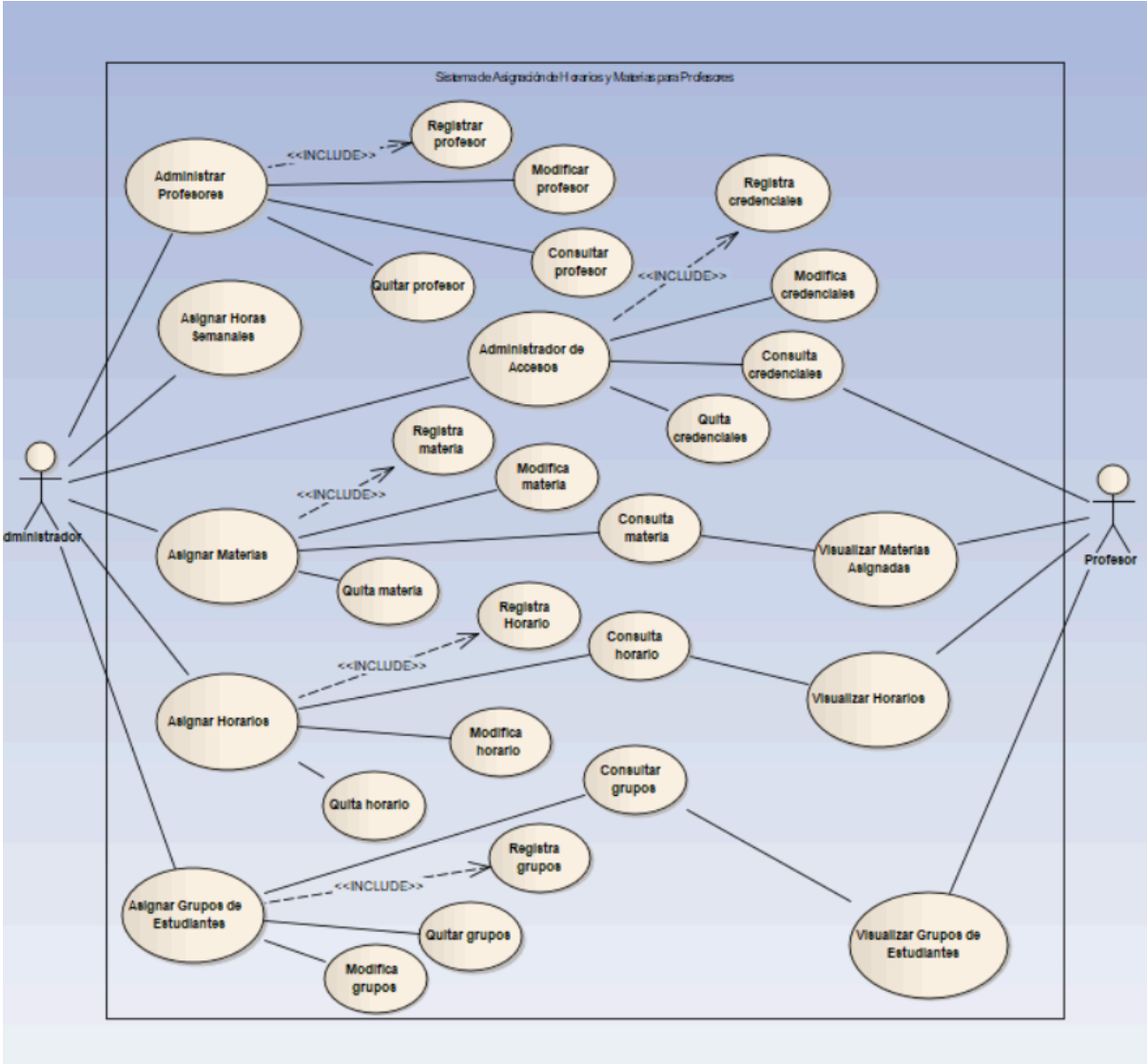


DIAGRAMA DE CASOS DE USOS



Versión	Fecha	Realizado por	Tiempo requerido
0.1	27/02/25	Cuapantecatl Ruiz Raul Irving	3:00 hrs

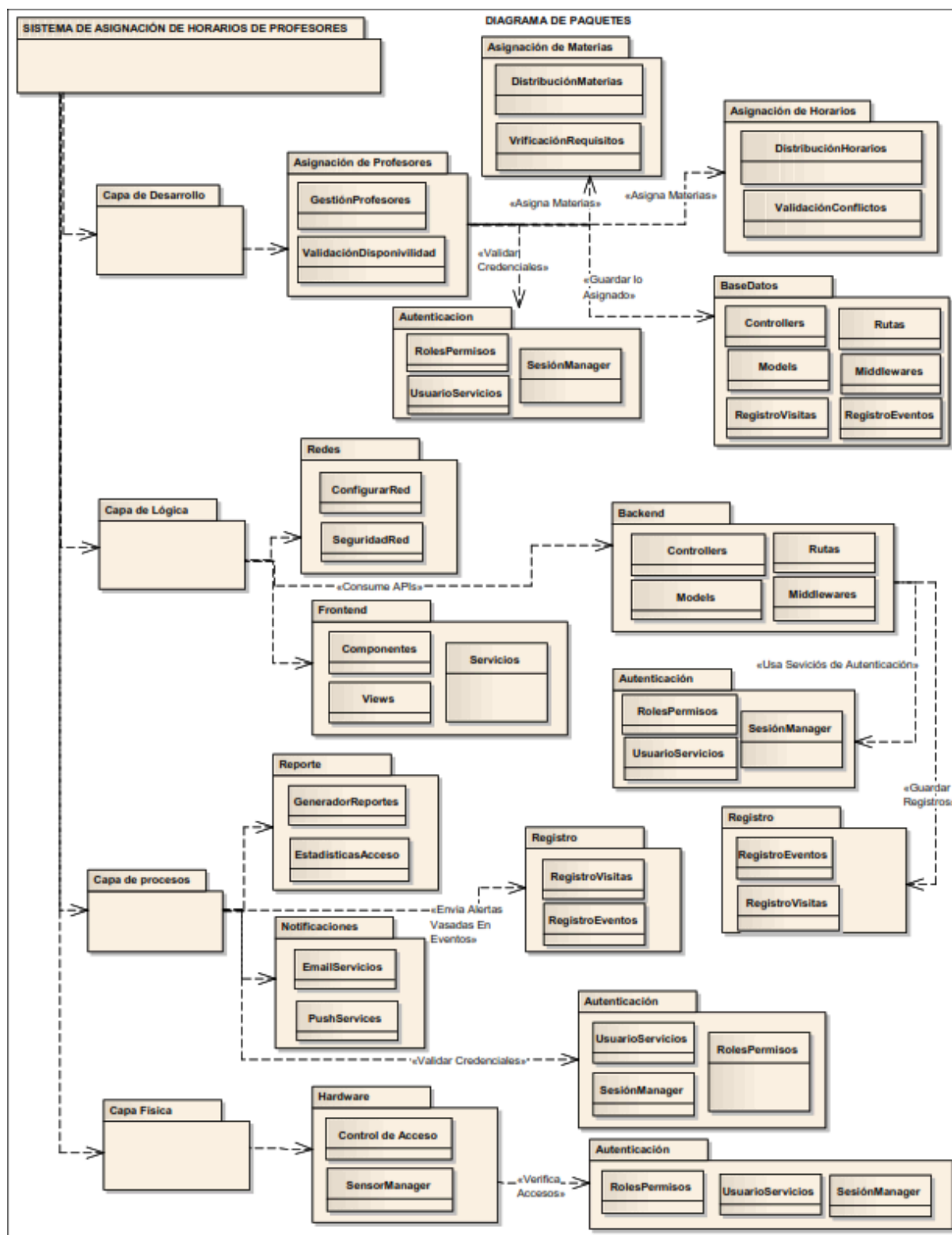
Composición:

La composición del sistema se puede refinar en dos puntos de vista principales: lógico y físico.

Vista Lógica

La vista lógica se centra en la funcionalidad y la estructura del sistema desde una perspectiva de diseño.

- Diagrama de Paquetes UML:
 - Este diagrama mostrará la organización lógica del sistema en paquetes. Se identificarán paquetes como:
- | | |
|---|---|
| <ul style="list-style-type: none">● Capa de Desarrollo<ul style="list-style-type: none">○ Clase Componentes○ Clase Views○ Clase Servicios● Redes<ul style="list-style-type: none">○ Clase Configurar Red○ Clase Seguridad de Red● Autenticación<ul style="list-style-type: none">○ Clase Usuario Servicios○ Clase Sesión Manager○ Clase Roles y Permisos● Registros<ul style="list-style-type: none">○ Clase Registro de Visitas○ Clase Registro de Eventos● Notificaciones<ul style="list-style-type: none">○ Clase Email Servicios○ Clase Push Services● Reporte<ul style="list-style-type: none">○ Clase Generador de Reportes○ Clase Estadísticas de Acceso● Capa Física<ul style="list-style-type: none">○ Clase Control de Acceso○ Clase Sensormanager | <ul style="list-style-type: none">● Asignación de Materias<ul style="list-style-type: none">○ Clase Distribución de Materias○ Clase Verificación de Requisitos● Asignación de Horarios<ul style="list-style-type: none">○ Clase Definición de Horarios○ Clase Validación de Conflictos● Asignación de Profesores<ul style="list-style-type: none">○ Clase Gestión de Profesores○ Clase Validación de Disponibilidad● Capa de Lógica● Base de Datos<ul style="list-style-type: none">○ Clase Controllers○ Clase Modelos○ Clase Rutas○ Clase Middlewares○ Clase Registro de Visitas○ Clase Registro de Eventos● Frontend |
|---|---|



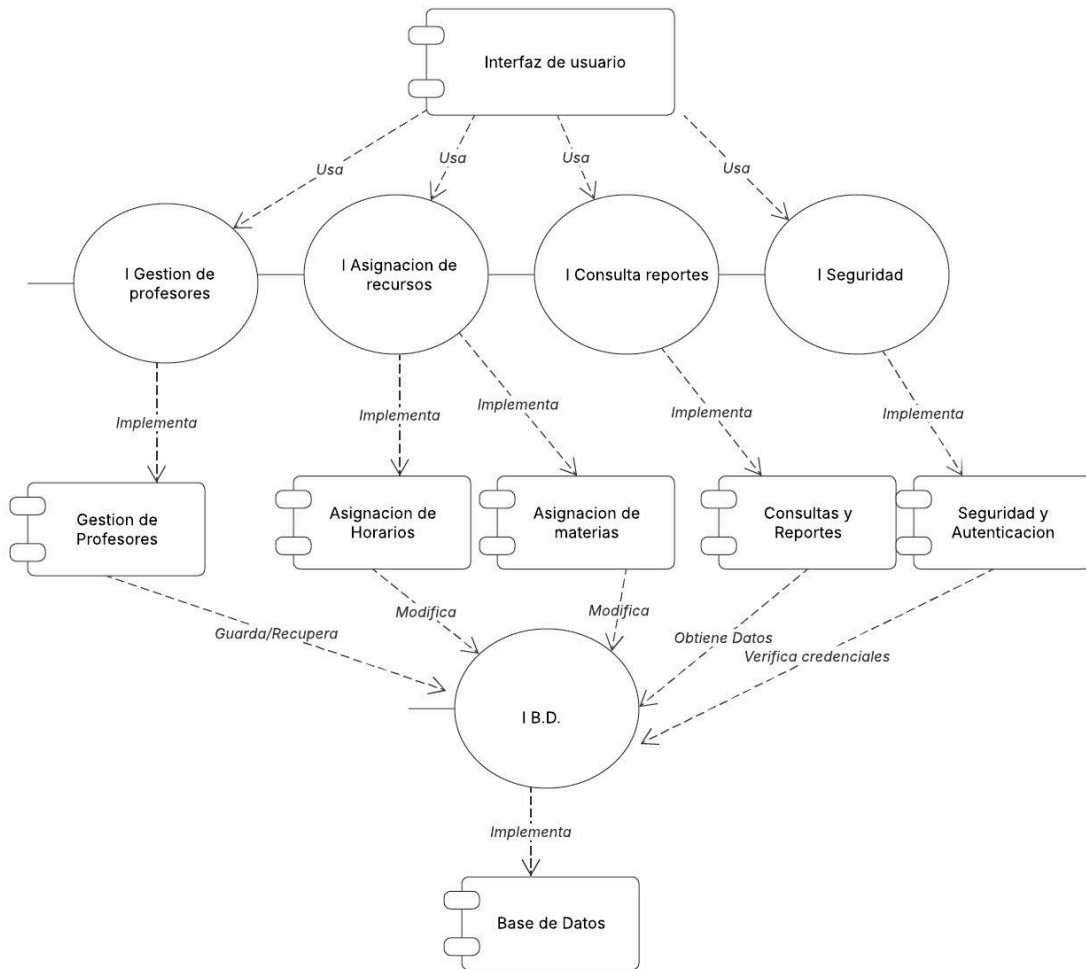
Este diagrama mostraría las dependencias entre los paquetes.

Versión	Fecha	Realizado por	Tiempo requerido
0.1	14/03/25	Cuapantecatl Ruiz Raul Irving	5:00 hrs

- Diagrama de Componentes UML:
 - Este diagrama detalla los componentes de software del sistema y sus interfaces. Se identificaron componentes como:
 - **Componente Profesores:** Responsable de las operaciones de administración de profesores.
 - **Componente Materias:** Responsable de las operaciones de gestión de materias.
 - **Componente Horarios:** Responsable de las operaciones de gestión de horarios.
 - **Componente Consultas y Reportes:** responsable de las operaciones de consulta de grupos de estudiantes, horarios y materias.
 - **Componente Interfaz:** Responsable de la interacción con los usuarios.
 - **Componente BaseDatos:** Responsable del acceso a la base de datos.
 - **Componente Seguridad y Autenticación:** Responsable del acceso al sistema.

Diagrama de componentes

Miguel Angel García León | March 13, 2025

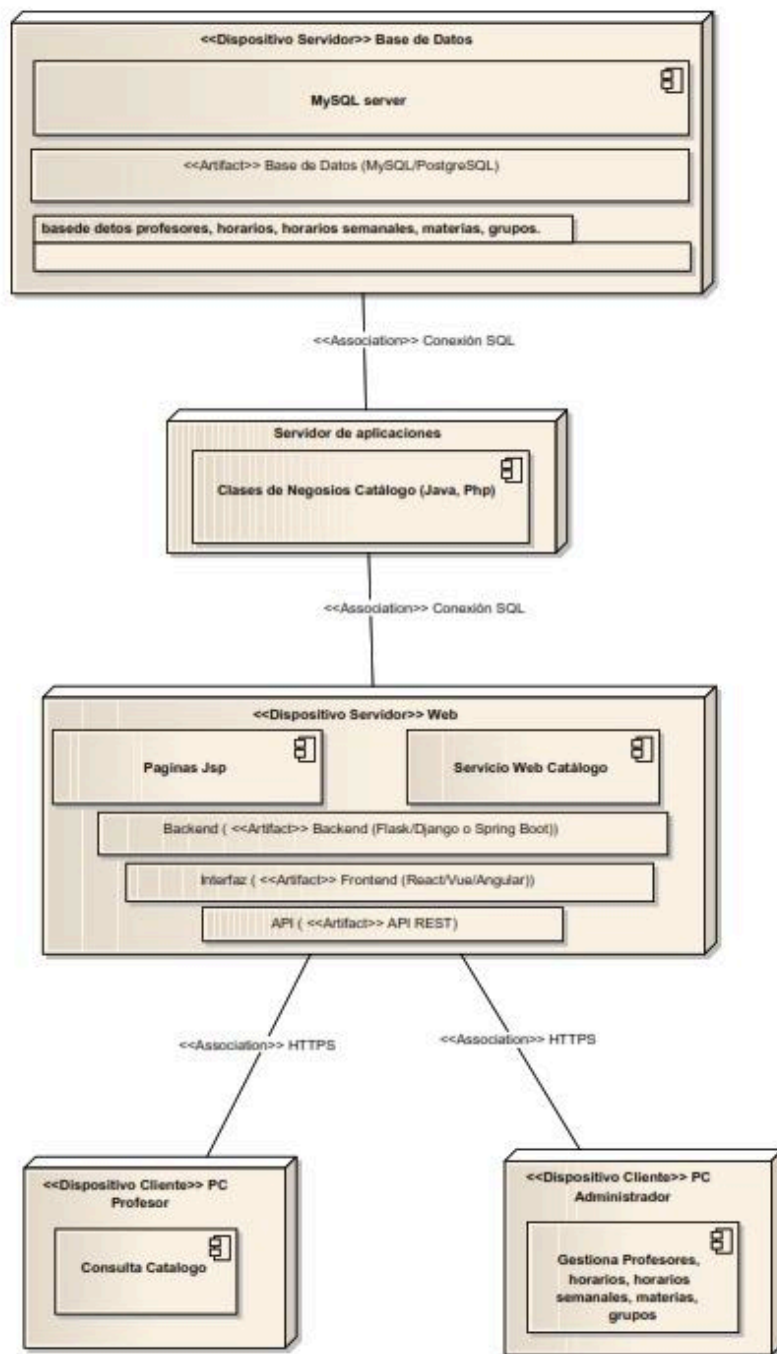


- Este diagrama mostraría las dependencias y las interfaces proporcionadas/requeridas por cada componente.
- Lenguajes de Descripción de Arquitecturas (ADL):
 - Estos lenguajes se podrían usar para describir formalmente la arquitectura del sistema, incluyendo los componentes, sus interfaces y las interacciones entre ellos.

Vista Física

La vista física se centra en la implementación y el despliegue del sistema en el entorno de ejecución.

- **Diagrama de Despliegue UML:**
 - Este diagrama mostrará los nodos físicos (servidores, dispositivos) donde se desplegarán los componentes del sistema. Se identificarían nodos como:
 - Servidor de Aplicaciones: Donde se desplegarán los componentes de la lógica de negocio.
 - Servidor de Base de Datos: Donde se desplegará la base de datos.
 - Estaciones de trabajo de los administradores y profesores.
 - Este diagrama mostraría la distribución de los componentes en los nodos y las conexiones de comunicación entre ellos.



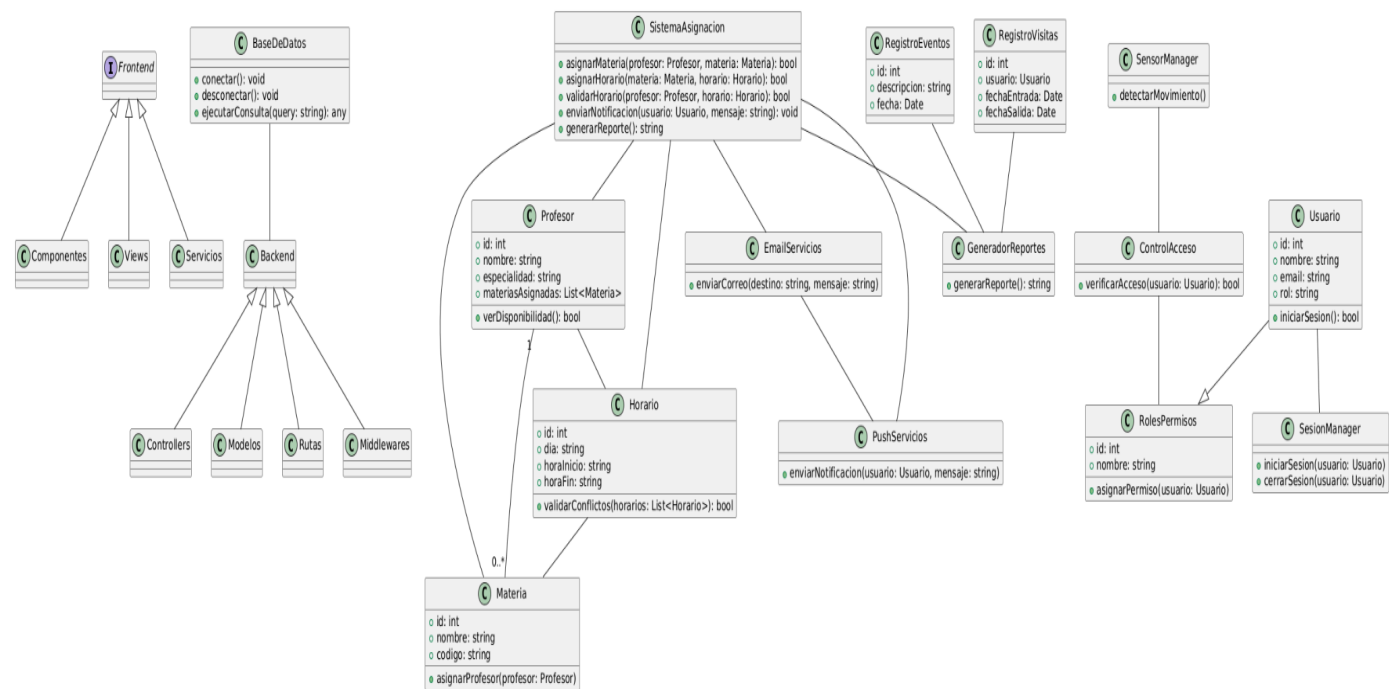
Versión	Fecha	Realizado por	Tiempo requerido
0.1	20/03/25	Cuapantecatl Ruiz Raul Irving	2:00 hrs

Diagrama de Clases (UML Class Diagram)

El **Diagrama de Clases** representa la estructura estática del **Sistema de Asignación de Horarios de Profesores**, incluyendo las clases principales, sus atributos, métodos y relaciones entre ellas.

Se han modelado clases como **Profesor**, **Materia**, **Horario**, **Notificación** y **Reporte**, estableciendo sus asociaciones mediante relaciones como **herencia**, **composición** y **asociaciones directas**.

- Punto de Vista de Composición:** Modela la estructura interna del sistema, identificando sus principales componentes y relaciones mediante diagramas de clases UML.



Versión	Fecha	Realizado por	Tiempo requerido
0.1	20/03/25	Garcia Leon Miguel Angel	4:00 hrs

Estos puntos de vista se utilizarán para garantizar que el diseño del sistema sea comprensible, estructurado y alineado con los requisitos funcionales y no funcionales previamente definidos.

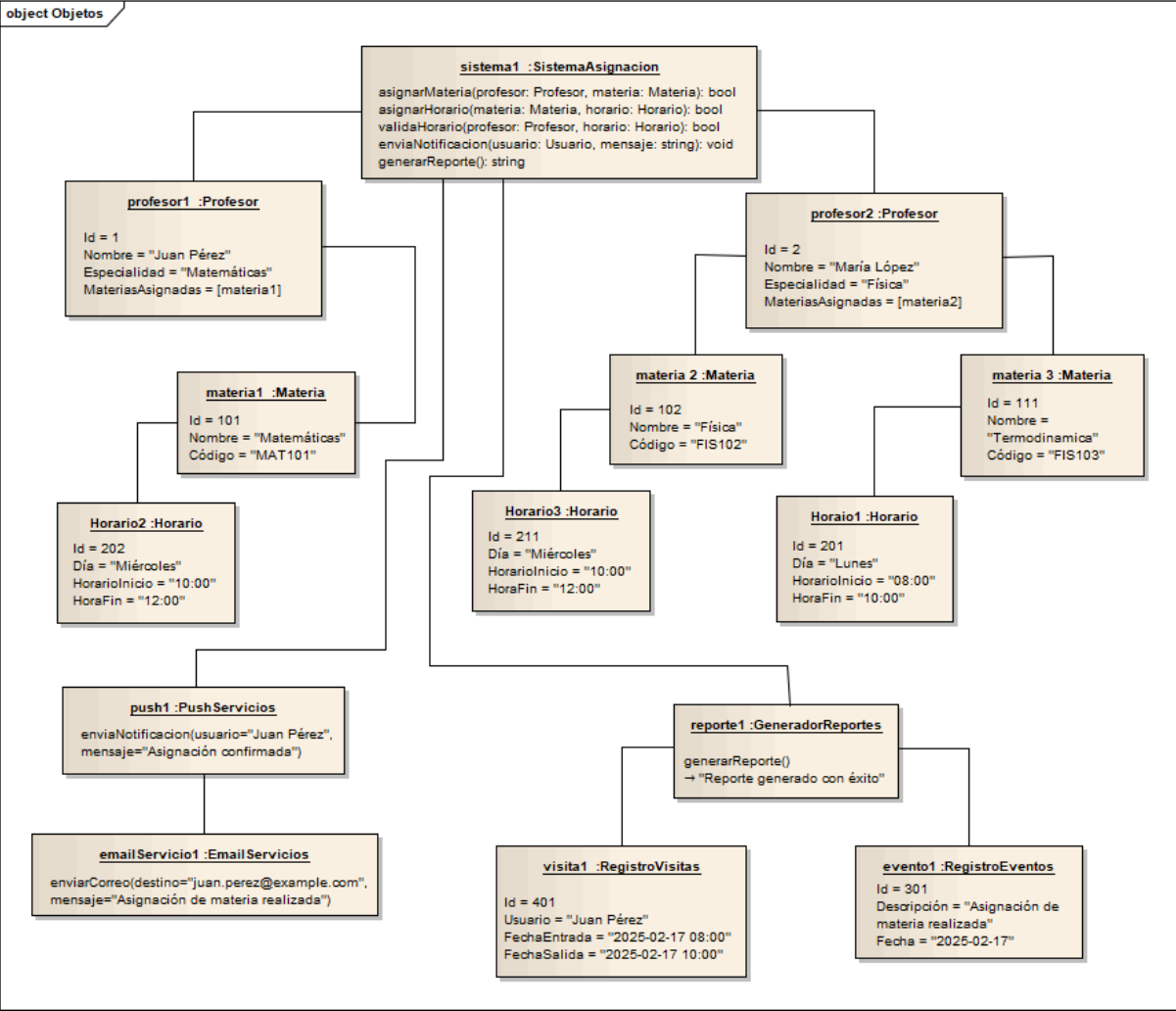
Diagrama de Objetos (UML Object Diagram)

El **Diagrama de Objetos** ejemplifica instancias concretas de las clases definidas en el **Diagrama de Clases**, mostrando un **estado específico** del sistema en ejecución.

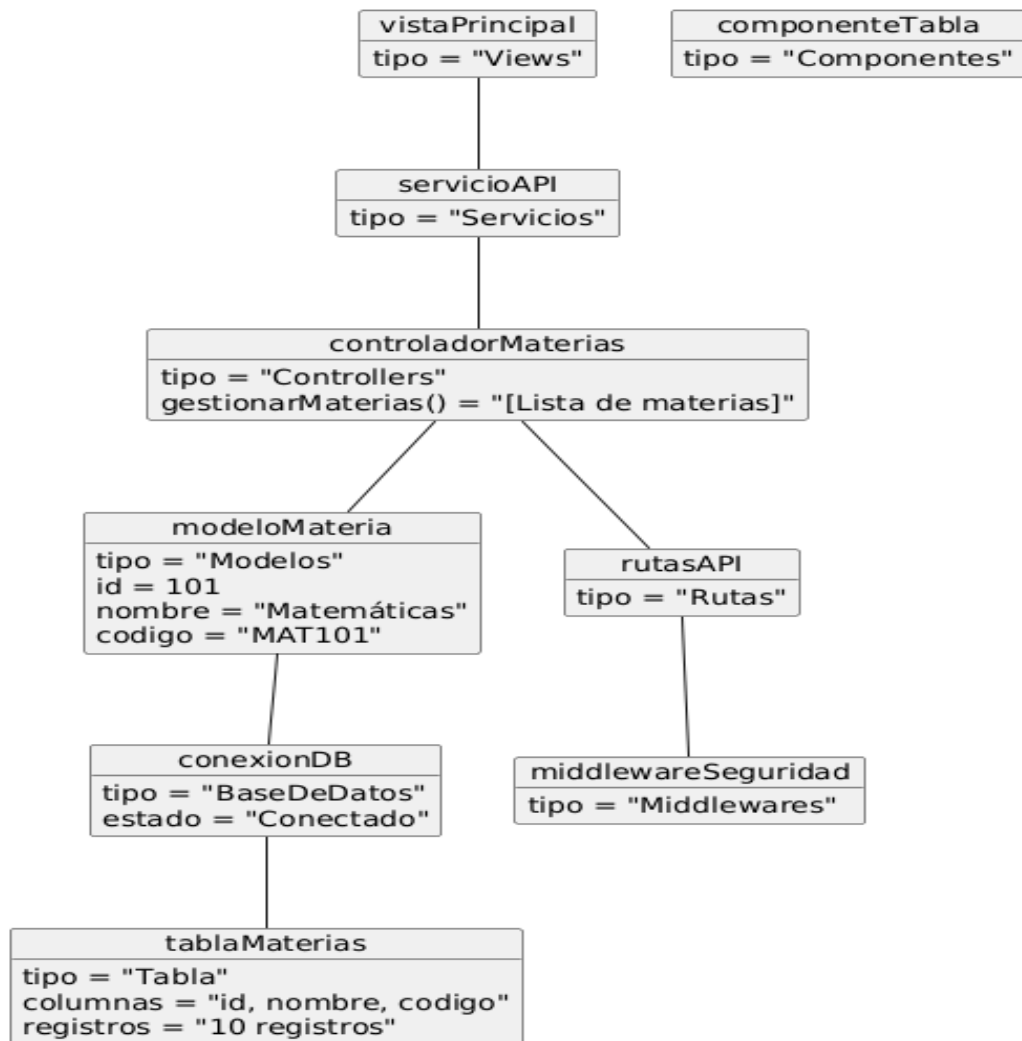
Por ejemplo, se presentan instancias como **Profesor: Juan Pérez**, **Materia: Matemáticas**, **Horario: Lunes 8:00 - 10:00**, y **Notificación: "Cambio de aula"**.

Relación con Logical (5.4)

- **Estructura estática (class, interfaces, and their relationships)**: Muestra un escenario real de cómo los objetos interactúan dentro del sistema.
- **Reuse of types and implementations (class, data types)**: Las instancias reutilizan estructuras definidas en el **Diagrama de Clases**, ejemplificando su uso práctico.



Versión	Fecha	Realizado por	Tiempo requerido
0.1	22/03/25	Cuapantecatl Ruiz Raul Irving	5:00 hrs

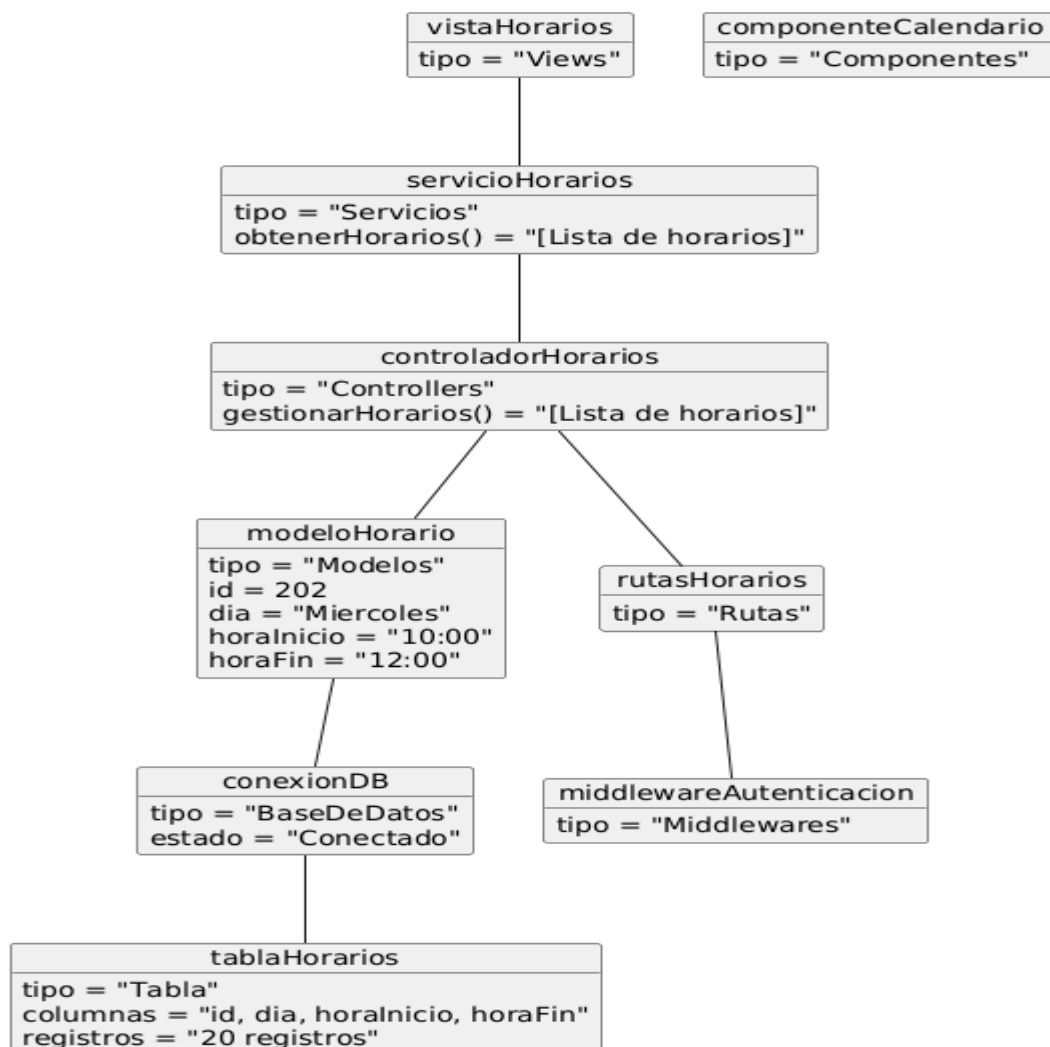


Versión	Fecha	Realizado por	Tiempo requerido
0.1	23/03/25	García Leon Miguel Angel	3:00 hrs

El diagrama de objetos para la gestión de materias representa cómo los diferentes elementos del sistema interactúan para asignar materias a los profesores. En este modelo:

- El Frontend cuenta con una vista específica para la gestión de materias y un servicio que obtiene la información desde el Backend.

- El Backend contiene un controlador encargado de gestionar las materias, las cuales se representan con objetos del modelo correspondiente.
- La Base de Datos almacena la información de las materias en una tabla específica y se gestiona a través de una conexión establecida.
- La comunicación entre estos elementos se realiza mediante rutas y middleware de autenticación, asegurando el acceso adecuado.



Versión	Fecha	Realizado por	Tiempo requerido
0.1	23/03/25	Garcia Leon Miguel Angel	3:00 hrs

El diagrama de objetos de gestión de horarios se centra en cómo se administran y validan los horarios asignados a las materias y profesores. La estructura es similar al diagrama de materias, pero adaptada al manejo de horarios.

- En el Frontend, la vista de horarios obtiene la información a través de un servicio y la representa gráficamente mediante un componente de calendario.
- En el Backend, el controlador de horarios administra la lógica de asignación, validación de conflictos y recuperación de datos desde el modelo correspondiente.
- La Base de Datos maneja los registros de horarios a través de una tabla que almacena los días, horas de inicio y fin, y posibles conflictos de asignación.
- Se mantiene una estructura segura mediante rutas protegidas por middleware de autenticación.

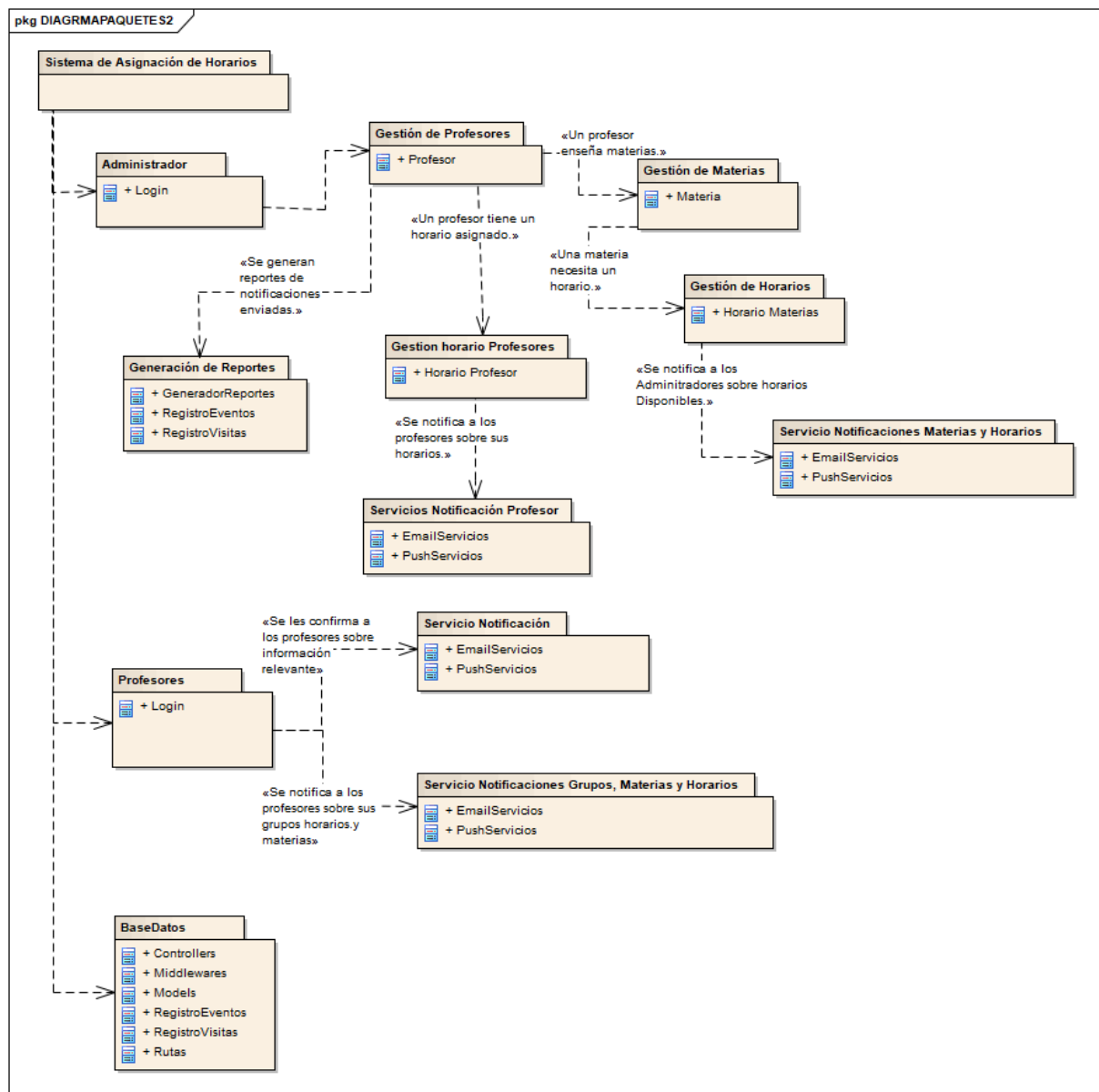
Dependency (5.5)

Diagrama de Paquetes (UML Package Diagram)

El Diagrama de Paquetes organiza el sistema en módulos lógicos y muestra sus dependencias.

Se han definido paquetes como Gestión de Profesores, Gestión de Materias, Gestión de Horarios, Servicios de Notificación y Generación de Reportes, estableciendo relaciones de dependencia entre ellos.

- Interconnection, sharing, and parameterization: Organiza el sistema en paquetes reutilizables y define las dependencias entre ellos.
- UML Package Diagram: Representa la agrupación lógica del sistema y su organización modular.



Versión	Fecha	Realizado por	Tiempo requerido
0.1	22/03/25	Cuapantecatl Ruiz Raul Irving	3:00 hrs

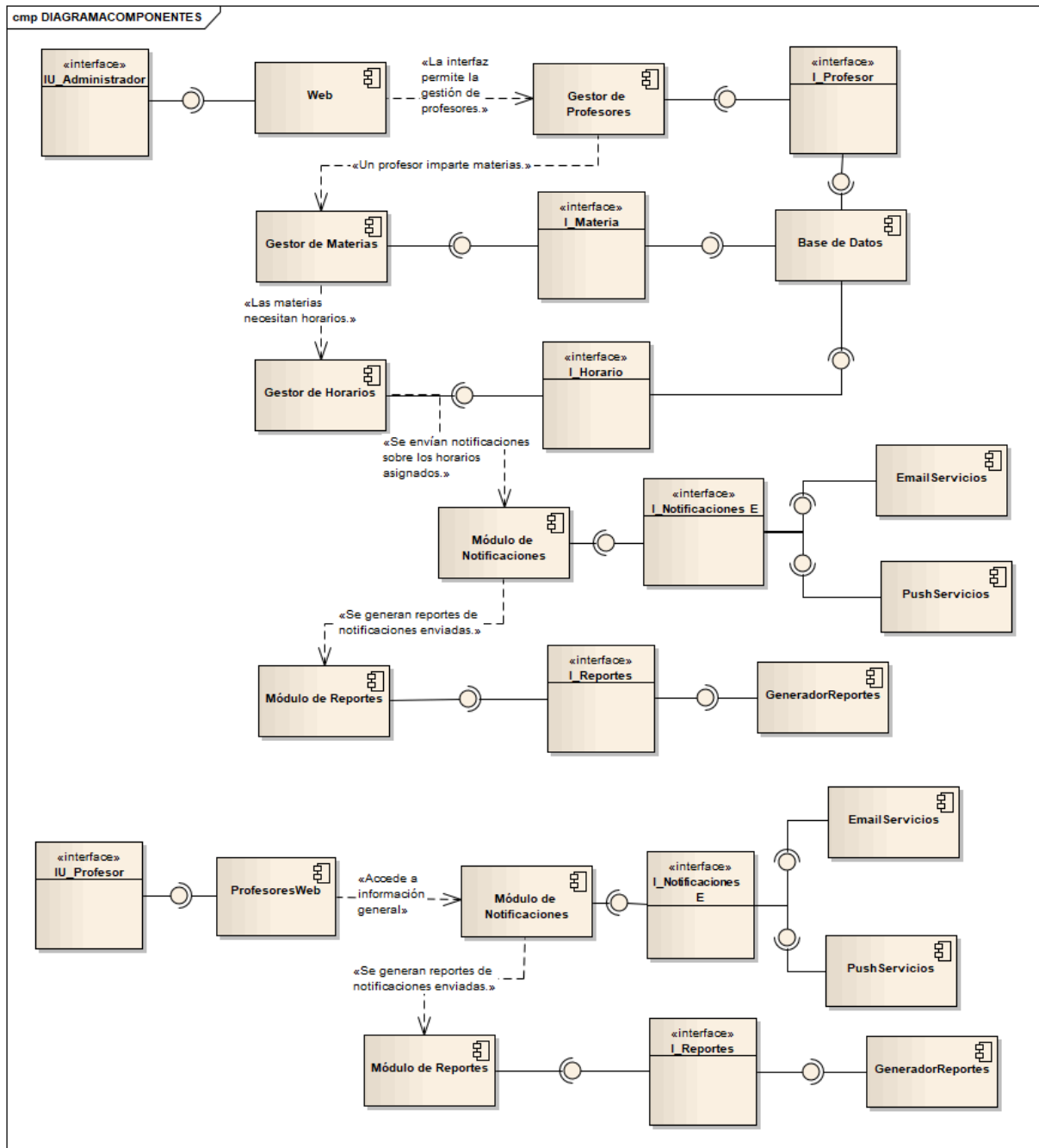
Diagrama de Componentes (UML Component Diagram)

El **Diagrama de Componentes** muestra la arquitectura física del sistema, definiendo los **módulos independientes** y sus interfaces de comunicación.

Se han modelado componentes como **Interfaz Web**, **Gestor de Profesores**, **Gestor de Materias**, **Gestor de Horarios**, **Módulo de Notificaciones**, **Módulo de Reportes** y **Base de Datos**, con sus respectivas **interfaces y dependencias**.

Relación con Dependency (5.5)

- **Interconnection, sharing, and parameterization:** Describe cómo los diferentes módulos del sistema están **interconectados** y comparten funcionalidades.
- **UML Component Diagram:** Representa la **arquitectura física** y los servicios que cada componente proporciona a otros.



Versión	Fecha	Realizado por	Tiempo requerido
0.1	22/03/25	Cuapantecatl Ruiz Raul Irving	4:00 hrs

Patterns (5.7)

1.- Patrón Strategy (Estrategia)

El Patrón Strategy permite definir un conjunto de algoritmos o estrategias para la asignación de horarios de profesores y seleccionarlos dinámicamente según las necesidades del sistema.

Implementación en el Sistema

Este patrón se aplicará en la funcionalidad de asignación de horarios, donde diferentes estrategias pueden ser utilizadas según criterios específicos como:

- Disponibilidad del profesor
- Carga horaria máxima permitida
- Preferencias de los profesores
- Restricciones institucionales

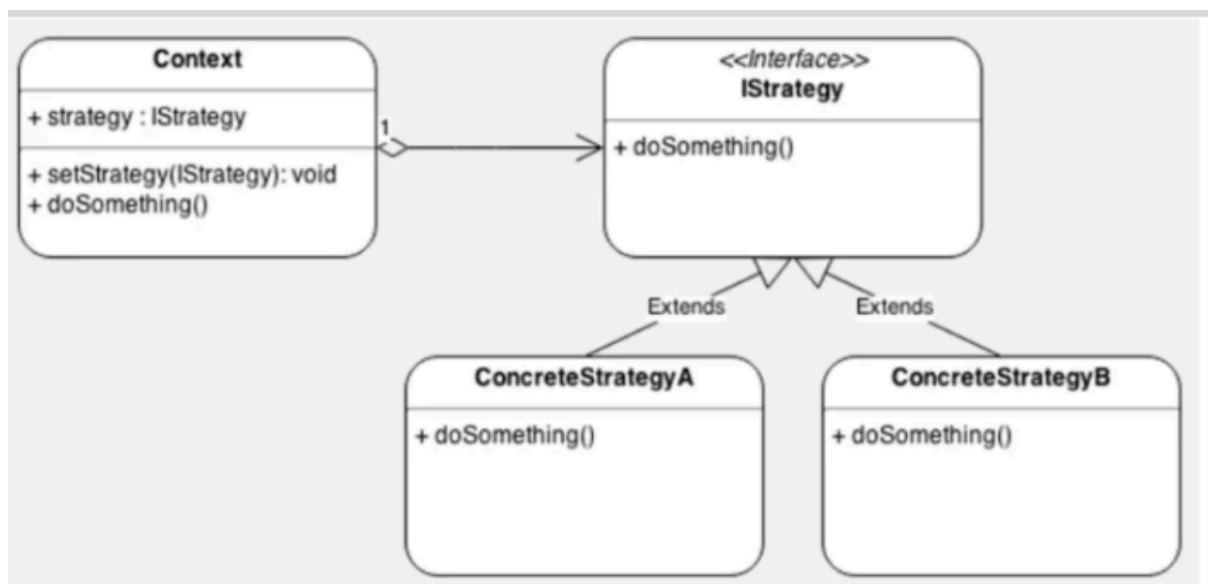


Imagen 1.1

Cada estrategia se implementará como una clase concreta que sigue una interfaz común, permitiendo su intercambio sin modificar la estructura del sistema.

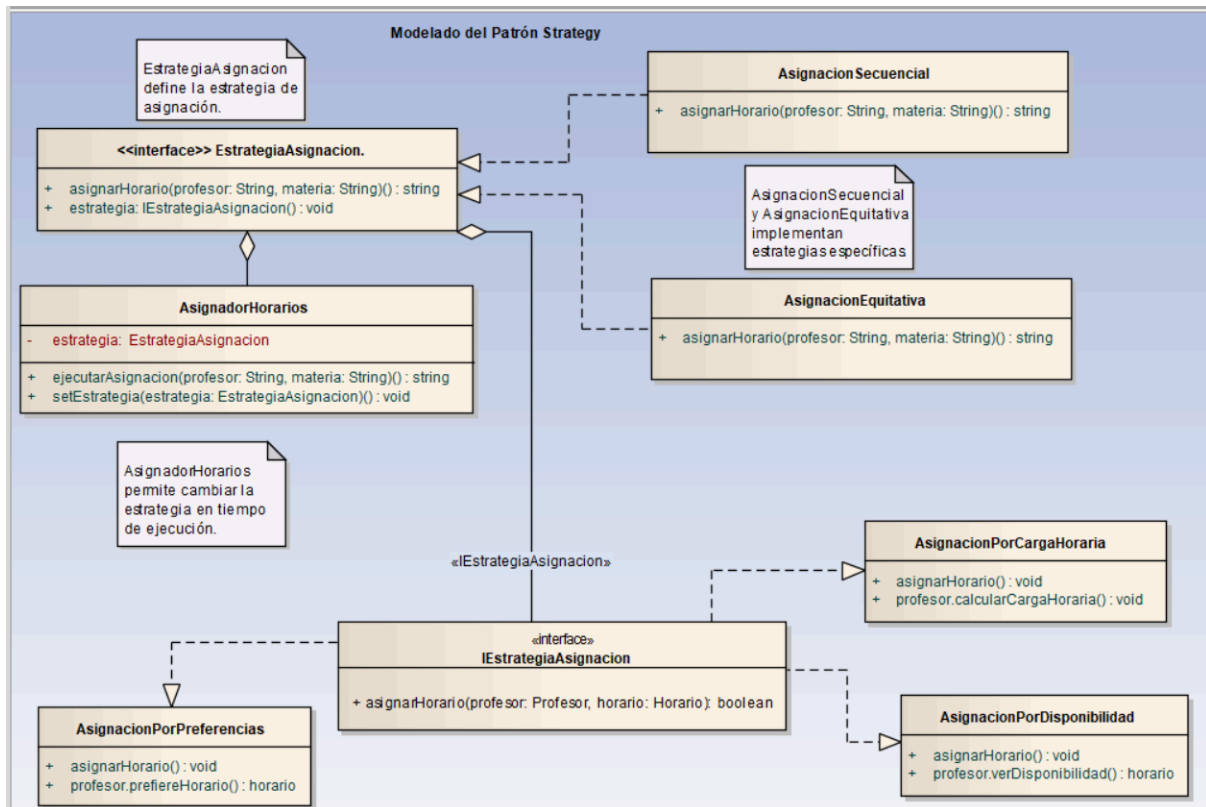


Imagen 1.2

Clase	Método	Descripción
AsignacionPorDisponibilidad	asignarHorario()	Verifica disponibilidad del profesor.
AsignacionPorCargaHoraria	asignarHorario()	Controla que no exceda la carga máxima.
AsignacionPorPreferencias	asignarHorario()	Verifica si el horario coincide con las preferencias.

Beneficios

- Permite definir múltiples estrategias de asignación.
- Mejora la flexibilidad del código al evitar estructuras rígidas.
- Facilita la integración de nuevas reglas sin modificar el núcleo del sistema
- Flexibilidad: Se pueden cambiar estrategias sin afectar el código existente.
- Escalabilidad: Permite agregar nuevas estrategias sin modificar las clases principales.
- Mantenimiento Simplificado: Evita grandes modificaciones en la lógica central.

- Personalización: Adapta la asignación de horarios según reglas específicas de la institución.

2.- Patrón Observer (Observador)

El Patrón Observer permite que los objetos interesados en un evento sean notificados automáticamente cuando ocurre un cambio en el estado del sistema.

Implementación en el Sistema

Se aplicará en la gestión de notificaciones cuando se realicen cambios en los horarios asignados. Ejemplo de uso:

- Si el horario cambia, los profesores y alumnos registrados recibirán una notificación.
- Se integrará con los servicios de EmailServicios y PushServicios para enviar notificaciones en tiempo real.

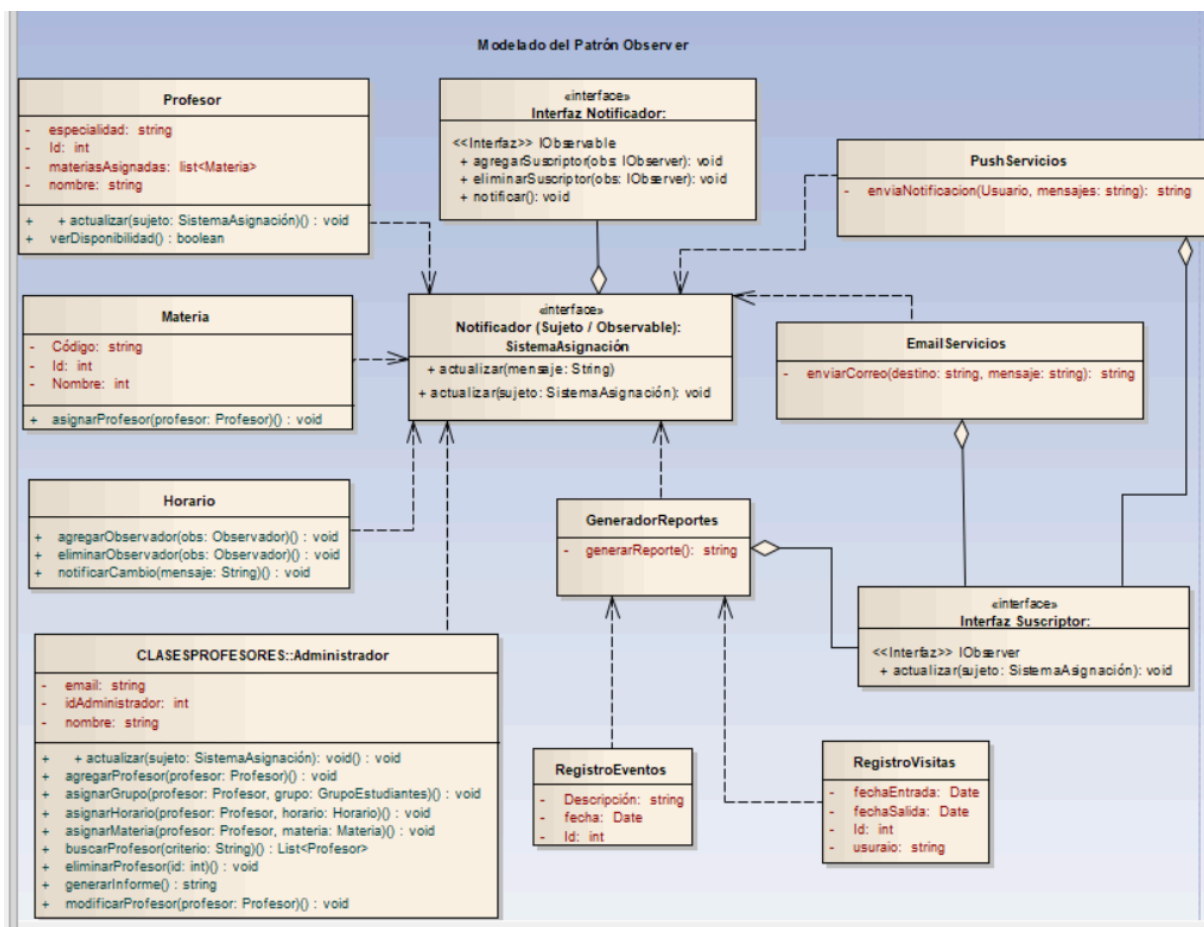


Imagen1.3

Lógica del patrón Observer aplicada:

Evento: asignarMateria o asignarHorario

1. SistemaAsignacion actualiza los datos.
2. Llama a notificar("Materia asignada a profesor X").
3. Cada suscriptor (EmailServicios, PushServicios, GeneradorReportes) reacciona vía actualizar.

Beneficios

- Facilita la comunicación automática de cambios en los horarios.
- Reduce la dependencia entre módulos mediante una arquitectura desacoplada.
- Mejora la escalabilidad del sistema al permitir nuevos suscriptores sin modificar el código base.

3.- Modelo-Vista-Controlador (MVC):

Un patrón que separa la aplicación en tres componentes interconectados: Modelo (datos y lógica de negocios), Vista (interfaz de usuario) y Controlador (maneja la entrada del usuario y actualiza el modelo y la vista). Componentes clave: Modelo, Vista, Controlador.



Implementación en el Sistema

El sistema de asignación de horarios utilizará el patrón **MVC** para separar la lógica de negocio de la presentación y la gestión de eventos.

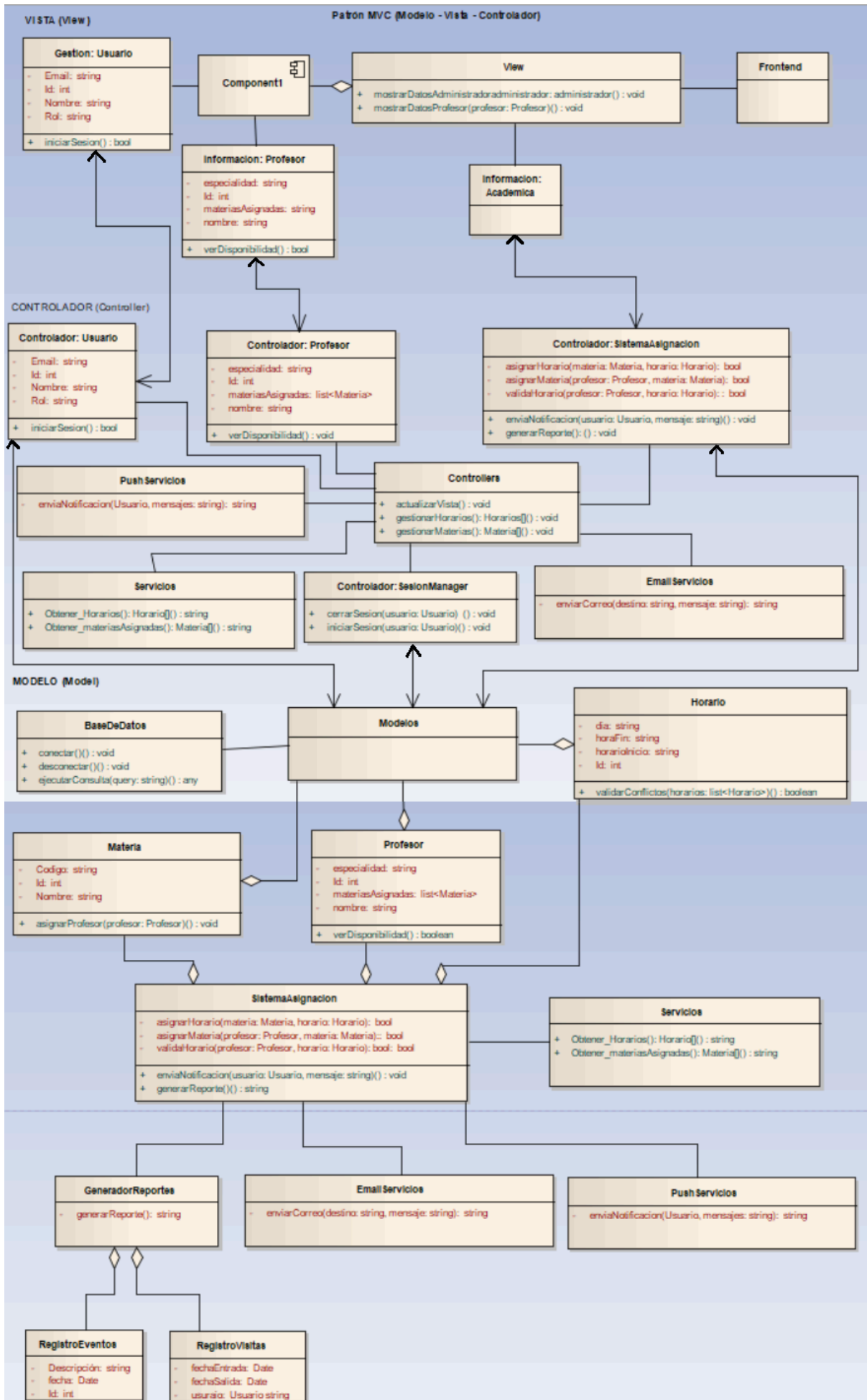


Imagen 1.4

Versión	Fecha	Realizado por	Tiempo requerido
Imagenes {1.1, 1.2, 1.3, 1.4} 0.1	11/04/25	Cuapantecatl Ruiz Raul Irving	8:00 hrs

El controlador se encarga de la comunicación entre la vista y el modelo, evitando dependencias directas entre ellos.

Beneficios

- Modularidad: Separa la lógica de negocio de la interfaz de usuario.
- Facilita el mantenimiento: Cambios en la vista no afectan al modelo y viceversa.
- Reutilización: Permite reutilizar el modelo en diferentes interfaces (Web, móvil, escritorio).

En resumen de Patrones en el Sistema.

Patrón	Descripción	Implementación	Beneficios
Strategy	Define estrategias dinámicas para la asignación de horarios.	Diferentes clases representan reglas de asignación.	Flexibilidad, facilidad de mantenimiento.
Observer	Notifica automáticamente cambios en horarios.	Profesores y alumnos son observadores del horario.	Desacoplamiento, escalabilidad.
MVC	Separa lógica de negocio, interfaz y controladores.	Modelo (datos), Vista (UI), Controlador (interacción).	Modularidad, reutilización, fácil mantenimiento.

Estos patrones permiten que el Sistema de Asignación de Horarios de Profesores sea escalable, flexible y fácil de mantener. Su implementación facilita la gestión eficiente de asignaciones, notificaciones y la interacción con el usuario.

Resumen

El objetivo es mejorar la gestión académica al automatizar la asignación de profesores a materias y horarios, asegurando una distribución justa y optimizada de los recursos. El sistema incluirá una interfaz web para administradores y profesores, permitiendo la gestión eficiente de horarios y evitando conflictos.

El sistema de asignación de materias y horarios está diseñado para universidades y otras instituciones educativas. Incluirá:

- Gestión de profesores: Registro, asignación, modificación y eliminación de docentes.
- Asignación de materias: Distribución de asignaturas según disponibilidad y especialización.
- Horarios: Definición de clases sin solapamientos.
- Gestión de aulas: Maximización del uso de espacios.
- Reportes: Generación de informes sobre carga de trabajo y distribución de materias.
- Seguridad: Control de accesos y protección de datos.

Para la implementación del sistema se requiere:

Infraestructura

- Servidor web para alojar la aplicación.
- Base de datos SQL (MySQL, PostgreSQL) para almacenar información.
- Red estable para acceso a la plataforma.

Software y Tecnologías

- Backend: Python (Flask/Django) o Java (Spring Boot).
- Frontend: HTML, CSS, JavaScript (React, Vue o Angular).
- Base de datos: MySQL, PostgreSQL o equivalente.
- Seguridad: Implementación de autenticación (JWT, OAuth).

Requisitos y Restricciones

- Acceso controlado según rol de usuario (administrador, profesor).
- Evitar colisiones de horarios entre profesores y materias.
- Uso eficiente de aulas para evitar espacios desaprovechados.
- Escalabilidad para soportar futuros aumentos de carga académica.

Enfoque Algorítmico

Para optimizar la asignación de profesores, horarios y aulas, el sistema implementará algoritmos como Gale-Shapley para la asignación de profesores y programación lógica condicional con restricciones para horarios y aulas.

Beneficios del Sistema

- Automatización de procesos administrativos.
- Reducción de errores y conflictos de horarios.
- Optimización del uso de aulas y profesores.
- Escalabilidad para futuras expansiones.

Se ha determinado que para lograr una distribución eficiente de los recursos académicos, se empleará un enfoque basado en algoritmos de optimización.

Objetivos

- Garantizar una distribución equitativa de las materias entre los profesores.
- Evitar solapamientos de horarios en las asignaciones.
- Maximizar la utilización de las aulas.
- Implementar un sistema escalable y flexible para futuras modificaciones.

Algoritmos Seleccionados

Para cumplir con los objetivos planteados, el sistema utilizará una combinación de algoritmos:

1. Algoritmo de Emparejamiento Estable (Gale-Shapley)

- Se utilizará para asignar materias a profesores de manera justa y eficiente.

- Garantiza que cada profesor reciba asignaciones acorde a su disponibilidad y especialización.
- Considera preferencias y restricciones para una distribución óptima.

2. Programación con Restricciones (Constraint Programming)

- Define reglas estrictas para evitar colisiones de horarios.
- Asegura que una clase no pueda ser asignada a más de un aula o profesor a la vez.
- Permite ajustar restricciones de capacidad y disponibilidad de aulas.

Enfoque Algorítmico

El sistema implementará algoritmos para optimizar la asignación de profesores, horarios y aulas.

- **Gale-Shapley:**
 - Este algoritmo se utilizará para la asignación de profesores a materias, teniendo en cuenta las preferencias de los profesores y las necesidades del sistema.
 - Aplicación: se aplicará para la asignación de profesores a materias, tomando en cuenta las especializaciones de los profesores, y las necesidades de cada materia.
- **Programación Lógica Condicional con Restricciones:**
 - Este enfoque se utilizará para la asignación de horarios y aulas, teniendo en cuenta restricciones como la disponibilidad de profesores, la capacidad de las aulas y las preferencias de los profesores.
 - Aplicación: se aplicará para la asignación de los horarios, tomando en cuenta la disponibilidad de los profesores, que no se sobre pongan los horarios de un mismo profesor, y tomando en cuenta que la materia asignada corresponda a la especialización del profesor.

Aplicación del Enfoque Algorítmico

- Para la asignación de profesores a materias (CU03), el algoritmo Gale-Shapley se aplicará para encontrar una asignación estable que maximice la satisfacción de los profesores y las necesidades del sistema.
- Para la asignación de horarios (CU04), la programación lógica condicional con restricciones se aplicará para encontrar una solución que cumpla con todas las restricciones y optimice la utilización de los recursos.
- para la asignación de grupos de estudiantes (CU05) Se tomara en cuenta las restricciones de los horarios de los profesores, y la disponibilidad de los grupos, para que no se generen conflictos.

Este enfoque algorítmico permitirá al sistema realizar asignaciones eficientes y justas, optimizando la utilización de los recursos y maximizando la satisfacción de los usuarios.

FALTA INDICAR EJEMPLOS.

Flujo de Asignación

1. Recolecta Información:

- Lista de profesores y materias.
- Disponibilidad horaria de los profesores.
- Capacidad y disponibilidad de aulas.

2. Fase 1 - Asignación de Profesores a Materias (Gale-Shapley):

- Se emparejan materias con profesores según su disponibilidad y especialidad.
- Se priorizan profesores con mayor experiencia en ciertas asignaturas.

3. Fase 2 - Asignación de Horarios y Aulas (Constraint Programming):

- Se evitan solapamientos de clases en horarios y aulas.
- Se buscan combinaciones óptimas para el uso de espacios disponibles.

Beneficios del Enfoque

- Evita conflictos de horarios y sobrecarga de trabajo en profesores.
- Maximiza la ocupación de aulas, evitando espacios vacíos.
- Permite escalabilidad para incluir nuevos profesores, materias y aulas.
- Reduce la carga administrativa en la asignación de horarios.

3.1 Diseño arquitectónico

3.2 Descripción de la descomposición

3.3 Justificación del diseño

4. DISEÑO DE DATOS

4.1 Descripción de los datos

4.2 Diccionario de datos

5. DISEÑO DE COMPONENTES

6. DISEÑO DE LA INTERFAZ HUMANA

6.1 Visión general de la interfaz de usuario

6.2 Imágenes de pantalla

6.3 Objetos y acciones en pantalla

7. MATRIZ DE REQUISITOS

8. ANEXOS

